Table of Contents

NEXUS AI — DOSSIER MAÎTRE DE SPÉCIFICATION ET DE DÉVELOPPEMENT

Version : 1.0 (base de travail détaillée)
Date : 10 octobre 2026
Statut : architecture proposée, à valider par tests de faisabilité
Produit : application desktop open source, local-first, principalement dédiée au codage assisté par des équipes d'agents IA.
Plateformes : Ubuntu/Linux (développement initial), Windows et macOS (publication).
Nom : « Nexus AI » provisoire ; disponibilité juridique, marques et domaines non vérifiée.

> **Mode d'emploi :** Ce document est la source de vérité du projet. Les identifiants FR-, NFR-, SEC-, ADR-, T- et AC- sont stables. Chaque agent de développement reçoit uniquement un paquet de contexte défini au chapitre 24, avec les contrats de son module. Les exigences marquées **MVP** sont obligatoires avant la première publication publique ; un prototype interne peut être incomplet, mais ne doit pas être présenté comme sécurisé.

1. Mission et vision

Nexus AI réunit dans une seule application des agents de programmation accessibles par CLI (Claude Code, Codex, Gemini CLI, Hermes et connecteurs futurs) et des modèles accessibles par API (OpenAI, Anthropic, Google AI Studio, OpenRouter, serveurs locaux et futurs fournisseurs). L'utilisateur crée un ou plusieurs projets, forme plusieurs équipes, choisit une gouvernance manuelle ou un superviseur IA, assigne des missions, observe les actions en temps réel et retrouve tous ses projets, conversations, tâches et décisions après la fermeture de l'application.

Différenciation visée, non encore prouvée : plateforme réellement extensible API + CLI, gouvernance configurable, mémoire partagée vérifiable, suivi d'utilisation prudent, sécurité effective des processus, GitHub et Vercel intégrés, dictée vocale, assistance à l'architecture et au déploiement. L'existence de concurrents possédant individuellement plusieurs de ces fonctions interdit de revendiquer leur nouveauté sans validation de marché.

Utilisateurs prioritaires : développeur indépendant, consultant, mainteneur open source, équipe technique de petite taille. **Positionnement :** coding-agent workbench, et non chatbot généraliste. Les recherches, résumés et documents sont des fonctions au service du travail logiciel.

2. Décisions consolidées et limites du périmètre

ID	Décision	État
ADR-001	Application desktop Electron + React + TypeScript, local-	Retenue

ID	Décision	État
	first	
ADR-002	Boilerplate léger officiel Electron Forge Vite+TS ; interface React ajoutée séparément	Retenue ; voir §6
ADR-003	Orchestrateur métier TypeScript écrit dans Nexus AI ; ne pas cloner un orchestrateur concurrent	Retenue
ADR-004	SQLite locale, journal d'événements append-only et snapshots	Retenue
ADR-005	Conception en packages séparés, IPC typé, connecteurs plugin-like	Retenue
ADR-006	Une équipe peut être pilotée manuellement ou par un superviseur IA choisi	Retenue
ADR-007	Pair-à-pair autorisé pour demandes d'avis sous règles ; affectation/permissions centralisées	Proposition retenue pour MVP
ADR-008	Worktrees par tâche écrivant du code ; pas de merge silencieux	Retenue
ADR-009	Sandbox de processus et secret guard obligatoires dès MVP public	Retenue
ADR-010	Sessions persistantes ; reprise logique distinguée de reprise native CLI	Retenue
ADR-011	Intégration GitHub, preview Vercel, dictée vocale	MVP, avec conditions de faisabilité
ADR-012	Licence initiale Apache-2.0 ; stratégie de marque + services futurs	Recommandation, validation juridique souhaitable
ADR-013	Conseil IA facultatif ; déploiement automatique production soumis à validation	Retenue

Hors MVP : marketplace de plugins, SaaS multi-tenant, synchronisation multi-appareils, collaboration temps réel entre humains distants, télécommande mobile, déploiement générique vers tous les hébergeurs, agent entièrement autonome sans garde-fous, équivalence artificielle des quotas des fournisseurs.

3. Analyse concurrentielle et règles de réutilisation

Hydra — dépôt : https://github.com/jpdlr/hydra. Documentation : desktop multi-sessions Claude Code/Codex, terminaux PTY, sessions reprises, superviseur MCP, statistiques d'usage, budgets. **Forces :** interface d'observation, reprise, orchestration opérationnelle. **Limites pour notre objectif :** angle CLI particulier, architecture spécifique à son produit, compatibilité API universelle à ne pas présumer. **À apprendre :** gestion des terminaux, dashboard et cycle de vie. **À ne pas copier :** code ou design sans examen de licence et attribution.

Runner — https://github.com/yicheng47/runner. Documentation : équipes de rôles, mission, chef d'équipe, terminaux natifs, fil d'événements persistant, demandes ask_human. Support indiqué macOS Apple Silicon et Windows x64, Linux non pris en charge à la date de consultation. **Forces :** mission/crew/feed, coordination concrète, séparation des rôles. **Limites :** compatibilité plateforme moins large ; approche native Rust/GPUI incompatible avec notre démarrage Electron. **À apprendre :** protocoles événements, règles d'escalade vers humain.

Codey — https://github.com/its-ahoh/codey. Documentation : agents Claude Code/Codex/OpenCode/pi, équipes, dispatch auto, mémoire par workspace, graphes de flux, voix, reprises, mode table ronde. **Forces :** proximité fonctionnelle avec notre vision ; configuration par rôle et graphes. **Limites :** application desktop macOS indiquée, maturité/robustesse à évaluer par essais ; ses fonctions invalidant toute revendication d'innovation exclusive. **À apprendre :** sémantique des graphes, séparation mémoire globale/projet et voix. **Ne pas réutiliser ses textes/prompts à l'identique.**

Emdash — https://github.com/generalaction/emdash. Documentation : Electron multiplateforme, agents CLI multiples, worktrees, revue des diffs, GitHub/issues/PR, SSH. **Forces :** isolation Git, flux PR et test/revue. **Limites :** coût de complexité du produit complet si cloné, impossibilité de présumer la sécurité de son isolation pour nos besoins. **À apprendre :** création/nettoyage des worktrees, revue, résolution conflits ; code open source sous conditions de sa licence.

Pivio — https://pivio.app/ (documentation commerciale à vérifier régulièrement). **Forces annoncées :** expérience unifiée et visibilité de l'utilisation. **Limites de l'analyse :** les mécanismes internes et capacités exactes ne sont pas vérifiés par audit de code. **À apprendre :** simplicité de configuration, affichage des limites de consommation.

Principe légal : s'inspirer de comportements et de flux utilisateur ne signifie pas copier des illustrations, logos, contenus, chaînes de prompts ou sources propriétaires. Avant tout copier-coller de code : vérifier licence, détenteur des droits, avis de copyright, NOTICE, dépendances, attribution, compatibilité Apache-2.0 et politiques de contribution.

4. Parcours utilisateurs (exemples exécutables)

UC-01 Première installation : ouvrir Nexus AI sur Ubuntu ; l'outil inspecte *en lecture seule* PATH pour claude, codex, gemini, hermes, opencode, git, gh, vercel; lit version via commandes non interactives sûres avec timeout ; expose installed, version, path, capabilities, authStatus (si test autorisé). Aucun téléchargement ou login automatique. Un bouton « Configurer » conduit à l'authentification officielle.

UC-02 Session manuelle : utilisateur charge un dépôt ; crée équipe « Architecte, Backend, Frontend, QA » ; assigne agent A au backend et B au frontend ; chacun peut lire les contrats, travaille en worktree isolé ; utilisateur voit tâches, messages, diff, logs, temps et budgets.

UC-03 Mode IA : utilisateur désigne Codex ou Claude comme « superviseur » ; demande « ajoute authentification et tests » ; superviseur propose un DAG, chaque tâche ayant fichiers cibles, entrées, sortie attendue, dépendances et critères ; règles vérifient autorisations et budgets ; exécution parallèle seulement si les chemins et dépendances le permettent ; intégration après review.

UC-04 Consultation pair-à-pair : agent Frontend envoie « quel contrat JSON de connexion ? » à Backend ; bus d'événements enregistre demande ; Backend répond avec schéma versionné ; le superviseur n'intervient pas tant qu'il n'y a ni nouvelle permission ni modification de périmètre ; messages visibles et filtrables.

UC-05 Retour après fermeture : quitter app ; la base conserve conversation, décisions, tâches, snapshots, sessions natives et dernière séquence d'événements ; au redémarrage l'interface réhydrate le projet. Une tâche arrêtée devient interrupted ; « reprendre » utilise le jeton/ID de session CLI si disponible, sinon crée un nouveau tour avec résumé contrôlé. Ne jamais afficher running sans processus actif vérifié.

UC-06 Github : voir diff ; sélectionner fichiers ; committer ; créer branche ; demander push ; écran de validation présente repo, branche distante, commits, fichiers, éventuels secrets détectés et effet ; utilisateur confirme. Mode automatique n'autorise pas d'emblée push/force push.

UC-07 Déploiement Vercel : détecter framework ; vérifier build et permissions ; montrer projet/équipe cible, variables requises et coût potentiel ; lancer vercel deploy pour preview après consentement ; collecter URL de preview et logs ; prod via confirmation distincte. Les identifiants ne sont pas exposés aux agents génériques.

UC-08 Dictée : appui micro explicite, vérification autorisation OS ; transcription locale si moteur compatible choisi, ou cloud après consentement ; l'utilisateur corrige le texte puis envoie ; audio supprimé ou conservé selon préférence. Ne jamais démarrer un enregistrement silencieusement.

UC-09 Protection .env : un agent demande cat .env ; broker refuse si chemin protégé même si la commande shell est reformulée ; sandbox n'a pas monté le fichier ; journal affiche le refus sans afficher son contenu. Un secret nécessaire au déploiement est utilisé par un connecteur spécialisé dans le périmètre autorisé, pas recopié dans le chat.

UC-10 Architecture : « Dessine l'architecture de mon API » ; l'agent analyse les fichiers autorisés, génère Mermaid, identifie services/dépendances et propose ADR ; rien n'est déployé sans décision séparée.

5. Exigences fonctionnelles (catalogue traçable)

ID	Priorité	Exigence et résultat observable
FR-001	MVP	Projet ouvrir/créer, associer dépôt Git local et chemin valide
FR-002	MVP	Plusieurs projets et plusieurs équipes sauvegardés
FR-003	MVP	Inventaire CLI démarrage + rafraîchissement manuel, sans auth intrusive
FR-004	MVP	Connecteur CLI Claude Code et Codex, priorité du premier jalon
FR-005	MVP	Adaptateur générique CLI ; Gemini CLI, Hermes suivant compatibilité testée
FR-006	MVP	Adaptateurs API OpenAI, Anthropic, Google et OpenRouter ; protocole unifié
FR-007	MVP	Support de plusieurs profils/comptes par fournisseur dans les limites autorisées
FR-008	MVP	Mode manuel ; création/affectation/annulation de tâches
FR-009	MVP	Superviseur IA choisi par utilisateur et routage capacités/coût/risque
FR-010	MVP	DAG de tâches et exécution simultanée dans worktrees distincts
FR-011	MVP	Messages inter-agents structurés et journal visibles
FR-012	MVP	Historique conversations/événements persistant et reprise contrôlée

ID	Priorité	Exigence et résultat observable
FR-013	MVP	Affichage sorties streaming, statut tâches, fichiers et erreurs
FR-014	MVP	Permissions manual, notify, sandbox-auto configurées
FR-015	MVP	Sandbox d'exécution réelle, pas seulement masquage GUI
FR-016	MVP	Secret guard, coffre credentials et redaction des sorties
FR-017	MVP	Git diff, commit, branch, GitHub auth et push confirmé
FR-018	MVP	Déploiement Vercel preview + confirmation production et logs
FR-019	MVP	Saisie vocale au choix + édition de transcription avant envoi
FR-020	MVP	Quotas/usage : réel, estimé ou indisponible, horodaté + source
FR-021	MVP	Plan d'architecture et diagramme Mermaid exportables
FR-022	MVP	UI explorateur/éditeur Monaco, terminaux, tâches, mémoire, activité
FR-023	MVP	Arrêt d'urgence : tuer processus et marquer tâches interrompues
FR-024	MVP	Paramétrage budget, timeouts, limites de boucles et nombre d'agents
FR-025	MVP	Import/export local d'un projet avec secrets exclus
FR-026	PLUS TARD	Conseil IA multi-propositions, activable sans obligation
FR-027	PLUS TARD	Connecteurs hébergement

ID	Priorité	Exigence et résultat observable
		VPS, Docker distant, autres clouds
FR-028	PLUS TARD	Collaboration humaine en ligne et partage distant des sessions

Priorités MVP ≠ livrables du premier soir. Le premier jalon interne couvre FR-001,003,004,008,010,011,012,013 en environnement restreint ; une release publique attend notamment FR-015 à FR-019 et tests plateforme.

6. Boilerplate, dépôts et stratégie de démarrage — décision précise

Boilerplate principal recommandé : Electron Forge officiel, template Vite + TypeScript : https://github.com/electron/forge ; guide : https://github.com/electron/forge/blob/main/docs/templates/vite-%2B-typescript.md . Commande de création à valider sur l'environnement de développement :

```
node --version
npm --version
npx create-electron-app@latest nexus-ai --template=vite-typescript
cd nexus-ai
npm start
```

Ce template ne fournit **pas automatiquement React** : ajouter React, ReactDOM et @vitejs/plugin-react selon la configuration Forge générée, puis construire les composants UI ; conserver les versions compatibles telles que figées dans le lockfile. Préférer npm pour la toute première exécution du template officiel, puis migrer explicitement à pnpm seulement quand les scripts Forge, builds et CI sont vérifiés. **Ne pas mélanger lockfiles.** Un pipeline CI teste le bootstrap sur Ubuntu.

Alternative évaluée, non retenue comme base initiale : https://github.com/electron-vite/electron-vite-react (React, Vite, Tailwind, Vitest/Playwright déjà assemblés). Atout : vitesse de démarrage ; risque : vérifier attentivement les réglages renderer/Node et reprendre le contrôle du packaging et des permissions. Peut être choisi à la place de Forge lors d'une ADR avant le premier commit, mais **pas combiné par copier-coller**.

Dépôts de référence à lire et non à cloner comme fondation : Hydra pour PTY ; Runner pour mission/feed ; Codey pour rôles/mémoire ; Emdash pour worktrees ; Pivio pour UX.
Dépôts d'intégration à étudier : https://github.com/cli/cli, https://github.com/vercel/vercel, https://github.com/microsoft/monaco-editor, https://github.com/xtermjs/xterm.js ; utiliser leurs packages officiels ou CLI plutôt que copier le code.

Bibliothèques proposées (vérifier versions/maintenance avant installation) : React, TypeScript, Vite, Tailwind, Monaco, xterm.js, node-pty, zod, better-sqlite3 ou sqlite native

robuste, vitest, Playwright, electron-log/pino, Mermaid, Git CLI, Electron Forge. **Ne jamais installer automatiquement tous ces modules le même jour :** ajouter par jalon selon besoin, pin et audit.

7. Architecture cible, frontières de confiance

```
Renderer React (UNTRUSTED UI)
   ⇅ IPC strictement typé via preload/contextBridge
Electron Main (fenêtres, dialogues, permissions, cycle de vie)
   ⇅ contrôleur local / service isolable
Core orchestrator (état, planificateur, politiques, budgets)
   ├── Event Store SQLite + snapshots + migrations
   ├── Registry connecteurs et inventaire CLI
   ├── Task DAG + scheduler + message bus
   ├── Sandbox Broker + Secret Broker + Process Supervisor
   ├── Git/Worktree Broker + GitHub Broker
   ├── Vercel Deploy Broker
   ├── Usage Meter / Voice transcription adapter
   └── CLI adapters (PTY/JSONL) ou API adapters (HTTPS)
         ⇅ exécution contrôlée et filtrée
      Agents et modèles externes / Git / environnement isolé
```

Règles : renderer sans accès Node direct ; contextIsolation=true, nodeIntegration=false, sandbox=true côté renderer compatible ; CSP restrictive ; aucune clé API envoyée au renderer ; toutes entrées IPC validées via schéma ; aucun exec('commande concaténée') ; préférer spawn(binary, args, {shell:false}). Sandboxing d'Electron **n'isole pas** les processus CLI ; ceux-ci passent par un runner séparé soumis à contrôles OS.

Local-first : aucun serveur central requis pour usage de base. Le CLI ou l'API choisi peut contacter un fournisseur externe selon ses propres règles. Le service runner peut être un processus enfant local distinct et, plus tard, un daemon ; journal SQLite garantit l'état durable, pas la survie magique d'une tâche quand l'app est fermée.

8. Arborescence cible du monorepo

```
nexus-ai/
  README.md  LICENSE  NOTICE  CONTRIBUTING.md  SECURITY.md
CODE_OF_CONDUCT.md
  package.json  package-lock.json  tsconfig.base.json  .gitignore
  docs/
    MASTER_SPEC.md  adr/  architecture/  api-contracts/  threat-model/
    runbooks/  user-stories/  agent-tasks/  testing/
  apps/
    desktop/
      electron/  src/  preload/  assets/  forge.config.ts
    runner/              # processus séparé pour orchestration et opérations risquées
```

```
packages/
  contracts/          # schémas Zod + types + versionnement
  core/               # modèle domaine, DAG, state machine, policy engine
  event-store/        # SQLite, migrations, snapshots, projections
  connectors/         # registry et interface commune
  connector-claude-cli/
  connector-codex-cli/
  connector-gemini-cli/
  connector-hermes-cli/
  connector-openai-api/
  connector-anthropic-api/
  connector-google-api/
  connector-openrouter-api/
  process-supervisor/   # spawn, PTY, backpressure, timeouts
  sandbox/            # backend Linux / macOS / Windows
  secrets/            # vault, brokers, masking
  git/                # branches, worktrees, review, conflict
  github/             # gh ou API, auth, PR, push
  deploy-vercel/      # CLI/API, preview, prod confirmée
  voice/              # capture, STT adapter, consentements
  usage/              # mesures sources / estimations
  observability/      # traces corrélées, redaction
tests/
  fixtures/  integration/  e2e/  security/  performance/
scripts/              # bootstrap, lint, packaging, diagnostic
.github/workflows/    # lint, test, package sur trois OS
```

Le générateur Forge produit initialement une arborescence mono-app : **étape T-002** déplace progressivement ses fichiers dans apps/desktop et configure les workspaces sans casser npm start. Aucun agent ne renomme des chemins partagés sans ADR et migration.

9. Contrat universel d'agent et de connecteur

```ts
export type AgentKind = 'cli' | 'api' | 'local';
export type Capability = 'chat'|'edit'|'terminal'|'structured-
output'|'resume'|'stream'|'usage'|'tools';
export interface ConnectorManifest {
  id: string; version: string; kind: AgentKind; displayName: string;
  capabilities: Capability[];
  supportedPlatforms: Array<'linux'|'win32'|'darwin'>;
  authModes: Array<'vendor-login'|'api-key'|'local-none'>;
}
export interface AgentTaskRequest {
  taskId: string; projectId: string; sessionId: string; agentProfileId: string;
```

```
  workspacePath: string; prompt: string; contextRefs: string[];
  permissionsPolicyId: string; timeoutMs: number; budget?: { tokens?: number;
currencyUsd?: number };
}
export type AgentEvent =
  | { kind:'started'; taskId:string; sequence:number; nativeSessionId?:string }
  | { kind:'output'; taskId:string; sequence:number; text:string;
channel:'stdout'|'stderr'|'message' }
  | { kind:'tool-request'; taskId:string; sequence:number; tool:string; payloadRef:string }
  | { kind:'usage'; taskId:string; sequence:number; source:'reported'|'estimated';
inputTokens?:number; outputTokens?:number; usd?:number }
  | { kind:'completed'; taskId:string; sequence:number; resultRef:string }
  | { kind:'failed'; taskId:string; sequence:number; code:string; message:string };
export interface AgentConnector {
  manifest: ConnectorManifest;
  probe(): Promise<{installed:boolean; version?:string; authenticated?:boolean;
reason?:string}>;
  start(req: AgentTaskRequest, signal: AbortSignal): AsyncIterable<AgentEvent>;
  resume?(nativeSessionId:string, req:AgentTaskRequest, signal:AbortSignal):
AsyncIterable<AgentEvent>;
  cancel?(taskId:string): Promise<void>;
  getUsage?(): Promise<UsageSnapshot>;
}
export interface UsageSnapshot {
  observedAt:string; source:'vendor'|'cli'|'local-estimate'|'unavailable';
  remaining?:number; consumed?:number; unit?:'tokens'|'requests'|'usd'|'percent';
  windowEnd?:string; caveat?:string;
}
```

Ce code est un **contrat de référence**, pas un fichier déjà compilé. Les implémenteurs stabilisent schémas Zod, sérialisation, erreurs, idempotence et tests dans packages/contracts avant de créer les adaptateurs. Un connecteur annonce uniquement ce qu'il sait réellement faire ; l'absence getUsage est normale.

CLI : processus non interactifs avec sorties structurées lorsque possibles (Claude --print --output-format stream-json, Codex exec --json sous versions compatibles) ; mode PTY pour interfaces interactives ; ne jamais supposer l'identité des formats d'événements ou l'autorisation d'utiliser un abonnement via un outil tiers. claude --resume et codex exec resume sont des capacités documentées, pas des garanties d'égalité de contexte.

API : fournisseurs SDK officiels lorsque pertinent, limites/erreurs normalisées ; OpenRouter et Google AI Studio via connecteurs dédiés ; clés dans vault seulement. Pour chaque fournisseur, tester authentification, coûts, streaming, outils, JSON, annulation et limites. Plusieurs comptes sont explicitement sélectionnés, sans usage qui viole les conditions fournisseur.

Découverte : registre de signatures (binaryNames, commandes --version, probeTimeout, capabilities) ; ne jamais parser des tokens secrets de dossiers d'auth ; ne pas promettre un pourcentage de quota si l'interface du fournisseur ne l'expose pas officiellement.

10. Gouvernance, protocole inter-agents et mémoire commune

Modes : MANUAL (l'humain attribue) ; AI_SUPERVISED (l'IA propose/attribue selon politique) ; ASSISTED (IA propose, humain valide le plan). Choix au niveau équipe et modifiable avec journalisation. Sélection du cerveau principal parmi les agents disponibles et compétents. Bascule superviseur : pause -> snapshot -> nouvelle attribution explicite.

Scheduler : découper mission en DAG ; exécuter seulement tâches dont dépendances satisfaites, budget disponible, permissions valides et conflits de chemin maîtrisés. États : draft -> ready -> queued -> running -> waiting_input|waiting_review -> completed|failed|cancelled|interrupted ; transitions transactionnelles et idempotentes.

Communication : enveloppe versionnée messageId, missionId, taskId, senderAgentId, recipientAgentId, type, payloadRef, timestamp, correlationId. Types task_assigned, question, answer, artifact_published, review_requested, review_result, decision_proposed, human_approval_requested, error. Le pair-à-pair ne donne jamais automatiquement de nouvelles permissions ni d'accès fichiers. Tout agent peut proposer une escalade ; seul le policy engine accorde.

Mémoire : couches (a) événements immuables, (b) état matérialisé, (c) faits et décisions de projet versionnés avec source/portée/confiance, (d) résumés compacts générés, (e) index de recherche optionnel ultérieur. Aucune mémoire « tout le texte de tous les agents à chaque prompt ». Chaque tâche reçoit un contexte déterministe : objectif, ADR pertinents, contrats, fichiers autorisés, derniers artefacts validés, décisions.

Exemple : Architect -> ADR-012 accepté -> API contract v3 publié -> Backend adopte v3 -> QA exécute tests v3 -> Reviewer signale incompatibilité -> ticket correction -> revalidation. Les agents voient références et versions, pas seulement un long chat fragile.

11. Persistance : modèle de données et reprise

Tables principales : projects, repositories, teams, agent_profiles, connections, provider_accounts (métadonnées seulement), sessions, messages, missions, tasks, task_dependencies, agent_runs, events, artifacts, file_changes, worktrees, approvals, permission_policies, usage_snapshots, deployment_runs, voice_transcripts, decisions, memory_entries, schema_migrations.

Champs minimum communs : UUID, created_at, updated_at si modifiable, version de schéma, références, statut enum. events a event_id, aggregate_id, sequence, occurred_at, event_type, payload_json_redacted, correlation_id. Unicité (aggregate_id, sequence) ; projection transactionnelle ; événement enregistré avant signal UI. agent_runs stocke native_session_id, connecteur/version, PID éventuel, état, timestamps. PID seul ne suffit pas pour reprendre après reboot.

Reprise : démarrage -> migration sauvegardée -> réparation projections par replay -> interrogation des processus encore vivants -> tous les running sans runner valide deviennent interrupted -> bouton Reprendre -> native resume si supportée, sinon nouveau run utilisant contexte reconstruit ; avertissement utilisateur pour actions non idempotentes. En fin de session, flush logs + sqlite checkpoint ; après crash, derniers événements déjà committés restent disponibles.

Sauvegarde : export versionné de projets/décisions/conversations avec secrets expurgés et chemins personnalisables ; restauration avec vérifications d'intégrité. Politique de rétention configurable, suppression vérifiable. Ne pas enregistrer le contenu des variables secrets dans messages, events, logs, traces ou exports.

12. Sandbox et protection des secrets — BLOQUANT MVP

Profil	Lecture	Écriture	Réseau	Actions sensibles
read-only	fichiers autorisés	interdite	refusé par défaut	refusées
sandbox-auto	arborescence montée autorisée	worktree de tâche	liste blanche explicite	broker et policy
notify	portée définie	portée définie	demande si nécessaire	confirmation
manual	portée définie	autorisation explicite	validation humaine	confirmation préalable

Architecture SEC-001 à SEC-008 : runner OS distinct, permissions minimales, montage/worktree autorisé seulement, fichiers secrets non montés, réseau par défaut limité selon politique, exécution shell avec arguments non concaténés, détection de chemins traversants/symlinks, limitation ressources et temps, redaction des logs, contrôle egress et appels outils, tests adversariaux. La politique du CLI seul n'est pas une barrière de sécurité suffisante. Si l'isolation OS n'est pas disponible, **bloquer les actions autonomes dangereuses** au lieu de simuler une sécurité.

Linux : prototype runner sous conteneur rootless ou bubblewrap avec namespaces et bindings restrictifs, sous réserve de support kernel et de tests ; prévoir fallback *refus* si mécanisme absent. **macOS :** étudier helper signé, séparation de processus et mécanismes système pris en charge ; ne pas supposer que le sandbox Electron s'applique aux CLI. **Windows :** étudier environnement isolé/WSL2 ou conteneurs compatibles, jeton restreint et ACL ; même objectif d'isolation vérifié par tests. Détails backend par OS devront être validés avant distribution, aucune équivalence présumée.

Secret Guard : credentials stockés dans coffre OS si disponible (Linux Secret Service, Windows Credential Manager, macOS Keychain) via intégration testée ; chiffrement/base secrète séparée seulement si nécessité ; jamais localStorage UI ; jamais injection globale d'un token API dans l'environnement de tous les agents. Les connecteurs GitHub/Vercel reçoivent les identifiants

via broker, sans exposer au modèle. Scanner diff avant push pour secrets connus ; refus par défaut si détection critique.

Modèle de menace : prompt injection depuis dépôt/issue, fichier .env, commandes destructrices, exfiltration par HTTP, symlinks hors sandbox, substitution de binaire PATH, extensions malveillantes, logs contenant token, scripts postinstall, erreurs de reprise, escalade agent-vers-agent. Pour chaque menace : test automatisé et procédure d'incident.

Critère absolu : pas de sortie MVP « sandbox disponible » tant que les tests de fuite/écriture/réseau et de confinement sur les trois OS cibles ne sont pas passés. Préversion Ubuntu interne possible avec avertissement explicite et autonomie limitée.

13. Git, GitHub et gestion des modifications

Git CLI via broker contrôlé ; 1 tâche écrivant du code = 1 worktree + branche isolée. Avant merge : diff, tests, lint, review et résolution de conflits ; changements non autorisés au worktree principal refusés. Les commandes de force push, suppression branche distante, nettoyage irréversible exigent approbation renforcée. Pour GitHub MVP, utiliser gh officiellement authentifié ou intégration OAuth/GitHub App validée ; **aucun mot de passe GitHub brut dans Nexus AI**. Fonctionnalités : détecter remote, statut, commit, push explicite, créer PR si CLI disponible, afficher URL/erreurs. gh auth login stocke habituellement en credential store, mais peut retomber sur fichier clair ; avertir l'utilisateur et ne pas le lire en brut.

Pipeline exemple : feature/login créée -> agent A modifie -> QA teste -> reviewer inspecte -> utilisateur accepte -> commit signé si disponible -> push -> ouvrir PR. Aucune modification distante silencieuse.

14. Vercel, hébergement et architecture visuelle

MVP Vercel : détecter package.json et framework, vérifier dépôt, construire, ouvrir connexion officielle vercel login si nécessaire, vercel link, vercel deploy preview, récupérer URL et logs, vercel deploy --prod après validation séparée ; parser correctement stdout/stderr et codes retour. Préférer commandes officielles et capacités déclarées ; nom de projet/domaine et environnements visibles avant exécution. Politique concernant fichiers .vercel, .env.local : ne pas remonter dans contexte IA ni committer. Le déploiement est un processus réseau privilégié et passe par un broker distinct.

Hébergement général : conseiller choix entre Vercel, VPS, conteneur, hébergement statique selon stack ; ne pas prétendre pouvoir déployer n'importe quel backend via Vercel.
Architecture diagrams : Mermaid flowchart, sequenceDiagram, diagrammes de services et README, avec prévisualisation locale et export texte/image ; l'IA documente hypothèses, coûts et limites.

15. Voice input et accessibilité

Flux : clic/maintien micro -> permission OS -> enregistrement indicateur visible -> moteur STT local sélectionnable si installé, cloud uniquement sur opt-in -> transcription éditable -> envoi par validation utilisateur. App ne transmet pas audio brut à un fournisseur non choisi.

Gestion erreurs absence micro, permissions refusées, langue FR/EN au minimum, sessions courtes, limite taille audio, interruption. Les moteurs concrets seront choisis après benchmark qualité FR, latence, packaging Linux/Windows/macOS, licence ; ne pas promettre fonctionnement vocal hors-ligne sans moteur installé.

Accessibilité : clavier pour les fonctions essentielles, lecteur d'écran, contrastes, captions/transcripts, arrêt facile, indicateur micro, interface i18n FR/EN.

16. Monitoring de consommation et budget

Trois classes : reported (fournisseur/CLI officiel), estimated (calcul local transparent), unavailable. Afficher horodatage, unité, période de réinitialisation *si fournie*, source et marge d'incertitude. Ne pas manipuler les endpoints privés/non documentés pour contourner les limites ni supposer les quotas d'abonnements équivalents aux crédits API. Budget par agent/projet, alerte avant seuil, stop à dépassement si calcul fiable ; sinon politiques conservatrices (temps/tours/tâches). Diagnostics anonymisables.

17. Interface utilisateur et pages

Navigation : onboarding/diagnostics ; projets ; espace projet avec explorateur de fichiers, éditeur Monaco, terminal xterm.js ; centre de missions ; vue équipes ; cartes agents en grille ; conversations ; mémoire et décisions ; review Git ; connexions/quotas ; déploiements ; paramètres sécurité ; historique/reprise. Un bouton visible « Arrêter les agents ». Réglage par équipe pour « IA superviseur / manuel / assisté », « Conseil IA on/off », « degré d'autonomie », « sandbox », « réseau ». Vue des conversations inter-agents affichant messages et actions observables ; **ne jamais promettre divulgation du raisonnement interne privé d'un modèle**.

18. Qualité, conformité, performance (NFR)

ID	Mesure cible initiale (à affiner par tests)
NFR-001	Ouverture interface sous 5 s sur machine de référence déjà provisionnée
NFR-002	Sauvegarde événement atomique avant accusé réception UI ; pas de corruption SQLite sous arrêt forcé
NFR-003	Contrats IPC validés et journalisés, 0 accès Node direct depuis renderer
NFR-004	Deux agents simultanés minimum sur machine adéquate
NFR-005	Arrêt/cancel lancé sous 3 s (hors processus OS non coopératif, escalade ensuite)
NFR-006	Tests unité >= 80 % sur core/policy (objectif, pas garantie de sûreté)
NFR-007	Installateurs Ubuntu, Windows, macOS testés en CI et en tests manuels

ID	Mesure cible initiale (à affiner par tests)
NFR-008	Pas d'identifiants, .env ou tokens en clair dans logs, exports et traces
NFR-009	100 % des push/production deploy soumis à garde d'autorisation explicite
NFR-010	Paramètres, labels UI, erreurs de confiance compréhensibles en FR/EN

19. Plan de validation et jeux d'essais (acceptance)

ID	Test d'acceptation
AC-01	CLI absent, présent non authentifié, présent authentifié détectés sans crash
AC-02	Deux providers différents collaborent et transmettent artefact versionné
AC-03	Concurrence : 2 worktrees isolés ; conflit simulé remonté sans écrasement
AC-04	Kill forcé pendant événement ; données relues ; session marquée interrupted
AC-05	Resume natif quand compatible ; fallback déclaré sinon
AC-06	.env, SSH keys et hors-workspace inaccessibles à agent sandboxé
AC-07	Tentative egress vers domaine non autorisé bloquée et enregistrée
AC-08	Confirmation requise avant GitHub push, PR ou Vercel production selon politique
AC-09	Clé API jamais présente dans export/log/UI/command line ou payload agent
AC-10	Quota inconnu affiché « indisponible » et pas 0 % ou 100 % fictif
AC-11	Audio consenti, transcription modifiable, aucun enregistrement silencieux
AC-12	Smoke tests installateurs Ubuntu/Windows/macOS et vérification des politiques
AC-13	Prompt injection issue GitHub « imprime secret » ignoré/bloqué
AC-14	Erreur réseau, limite fournisseur, binaire CLI non trouvé et version inconnue gérés

20. Séquencement impératif du développement (sans ambiguïté)

Phase 0 — Validation de faisabilité / ADR (T-000, T-001) : vérifier Ubuntu, Node, Git, CLI présents, compatibilité licence et versions ; tester POC spawn + stream JSON Claude/Codex sur dépôt factice ; POC sandbox Linux de lecture/écriture/réseau et stockage secrets. **Gate :** notes de test et décision technique écrites.

Phase 1 — Socle (T-002 à T-006) : initialiser Forge Vite TypeScript, ajouter React + IPC sécurisé, créer monorepo packages/contracts, SQLite migrations, CI lint/unit tests, diagnostics CLI. **Gate :** npm start affiche inventaire et projet enregistré ; redémarrage conserve état.

Phase 2 — Agents et mémoire (T-007 à T-011) : connecteurs Claude/Codex, exécution isolée, streaming, event store, conversation persistante, messages inter-agents, DAG manuel. **Gate :** deux agents réels ou un simulateur contrôlé échangent un artefact et reprennent après crash.

Phase 3 — Superviseur et sécurité (T-012 à T-016) : policy engine, rôles, routeur IA, budgets, secrets, sandbox linux puis backends OS, worktrees, audit. **Gate :** AC-02 à AC-07 et injection adversariale passent ; aucune exécution autonome hors confinement.

Phase 4 — Produit MVP (T-017 à T-022) : Monaco, terminal, GitHub, Vercel preview/prod, dictée, usage monitor, architecture diagrams, tests UI. **Gate :** scénario bout-en-bout app stock -> code -> tests -> review -> GitHub -> preview.

Phase 5 — Release cross-platform (T-023 à T-026) : backend sandbox Windows et macOS, packaging/signature, tests machines réelles, contrôle dépendances, documentation installation FR/EN, release GitHub. **Gate :** AC-01 à AC-14 et SEC testés sur trois OS ; sinon annoncer préversion Linux limitée.

Important : stages peuvent être construits parallèlement si contrats stabilisés ; ne pas lancer en parallèle deux équipes qui modifient les mêmes API sans coordination. Chaque T contient sortie vérifiable et PR.

21. Répartition de travaux aux IA : responsabilités de modules

Lot	Propriétaire IA suggéré	Chemins exclusifs	Dépendances	Preuve de livraison
A	Architecte (cerveau)	docs/adr/, packages/contracts	aucune	ADR + schémas validés
B	Core	packages/core/, packages/event-store/	A	tests machine états/migrations
C	Connecteurs	packages/connectors/, connector-*	A,B	fixtures, intégration simulée
D	Sécurité	packages/sandbox/, packages/secret	A	tests sandbox + menace

Lot	Propriétaire IA suggéré	Chemins exclusifs	Dépendances	Preuve de livraison
		s/		
E	Git/Deploy	packages/git/, github/, deploy-vercel/	A,D	tests sans production réelle
F	UI	apps/desktop/src/, preload/	A,B	E2E accessibilité
G	Voice/Usage	packages/voice/, packages/usage/	A,D	fixtures + consentement
H	QA/review	tests/, docs/testing/	tous	matrice AC et rapport

Intégration : les modifications de packages/contracts sont réservées à A ; tout autre lot propose une modification par issue/PR et attend approbation. Branches feat/<lot>-<ticket> ; travaux indépendants via worktrees. QA ne doit pas approuver son propre correctif sans vérification distincte.

22. Backlog d'implémentation en tickets copiables

Ticket	Entrées	Travail	Résultat attendu
T-000	Ubuntu	probe Node, Git, versions/SDK, matrice licences	docs/runbooks/environment.md
T-001	contrats CLI	POC streams + permissions + sandbox spike	docs/adr/001-runtime.md et tests
T-002	template Forge	bootstrap initial et verrouillage versions	desktop minimal lancé
T-003	sécurité Electron	preload API limitée, CSP, no nodeIntegration	E2E IPC refus invalides
T-004	structure	packages TS + build/test/CI	npm test vert
T-005	SQLite	migrations et event store	test de crash/replay
T-006	CLI discovery	registry + probes timeout	liste agent fiable
T-007	CLI adapters	Claude/Codex stream + lifecycle	traces normalisées
T-008	API adapters	OpenAI/Anthropic/	erreurs/budgets

Ticket	Entrées	Travail	Résultat attendu
		Google/OpenRouter	normalisés
T-009	sessions	conversations/events + resume	reprise sans perte confirmée
T-010	teams	équipes, rôles, méthodes manuelle	2 équipes configurées
T-011	bus	question/answer/ artifact event	dialogue inter-agent journalisé
T-012	scheduler	DAG, retries, deadlocks	tests dépendances
T-013	AI routing	plan JSON validé + policy	décision explicable
T-014	sandbox	Linux puis Windows/macOS	tests chemins/réseau
T-015	secrets	vault + redaction + broker	test fuite rouge/vert
T-016	worktrees	création, diff, review, merge	conflits détectés
T-017	UX code	Monaco, xterm, navigation	E2E utilisateur
T-018	GitHub	gh auth, status, push/PR	push test repo confirmé
T-019	Vercel	preview, prod approval, logs	preview URL vérifiée
T-020	voice	micro, STT, transcription review	dictée FR/EN
T-021	usage	snapshots et provenance	indicateur non trompeur
T-022	diagrams	Mermaid, exports	architecture projet générée
T-023	QA total	unit/integration/ e2e/security	matrice AC signée
T-024	packaging	Linux Windows macOS	installateurs smoke-testés
T-025	licence	Apache 2, NOTICE, marques	audit licences passé
T-026	release	docs install, release notes, support	version MVP publique

23. Règles de contribution et de définition de « terminé »

Chaque PR comprend : ticket ID ; objectif ; chemins modifiés ; tests exécutés et résultats ; impact sécurité ; éventuel changement de contrat ; instructions de rollback ; captures lorsque interface ; nouvelle doc ADR si décision. Statut DONE uniquement si compile, lint, tests, revue de sécurité et acceptation passent. Toute dépendance ajoutée doit mentionner licence et raison. Pas de nouvelles commandes externes sans permission broker. Pas de secret dans PR. Déployements réels hors CI de test soumis au propriétaire humain.

24. Paquets de contexte pour chaque IA (copier-coller)

Contexte global docs/agent-tasks/00_GLOBAL.md : vision §1, décisions §2, exigences §5, architecture §7, règles sécurité §12, modèle événements §10-11, définition de DONE §23. Aucune IA ne modifie contrats sans approbation. Ne pas inventer APIs, flags, quotas ou garanties de sandbox. Toujours annoncer hypothèses et tests manquants.

Template de ticket à fournir à une IA :

PROJET : Nexus AI. VERSION SOURCE : MASTER SPEC 1.0 (10-10-2026).
TICKET : T-___ ; LOT : ___ ; BRANCHE : feat/___ .
OBJECTIF : [une phrase vérifiable].
LECTURES OBLIGATOIRES : docs/agent-tasks/00_GLOBAL.md, docs/api-contracts/___, docs/adr/___, sections ___ de MASTER_SPEC.md.
CHEMINS AUTORISÉS : [...]. CHEMINS INTERDITS : [...].
CONTRATS ENTRÉE : [types + événements + versions].
SORTIES ATTENDUES : [fichiers, tests, docs].
TESTS OBLIGATOIRES : [commandes précises].
POLITIQUE : pas de réseau/deploy/push sans approbation ; ne jamais lire .env/secrets.
DÉFINITION DE TERMINÉ : compilation + tests + revue + rapport d'impact.
EN CAS D'AMBIGUÏTÉ : produire ADR/proposition, ne pas improviser contrat.
RETOUR : résumé, diff, tests, risques, décisions demandées.

Exemple Backend/EventStore : T-005, fichiers limités à packages/event-store/**, type AgentEvent fourni dans packages/contracts, migration 001_init, test arrêt forcé au milieu d'une séquence et replay exact. **Exemple UI :** T-017, afficher liste tâches à partir d'IPC missions:list sans lire directement SQLite. **Exemple sécurité :** T-014, prouver que processus exécuté ne peut pas ouvrir ~/.ssh/id_* ni écrire hors worktree ni contacter réseau non autorisé sur Ubuntu ; rapport d'échecs et contexte kernel.

25. Démonstration référence : « application de gestion de stock »

Input : « Construis une petite application de stock : connexion, catalogue produits (SKU, prix, quantité), entrées/sorties, alertes de seuil, tests ». Superviseur propose tâches : A modèle données/backend ; B interface produits ; C tests ; D intégration et review ; E diagramme et README ; F preview Vercel si compatible. Chaque tâche déclare chemins exclusifs, tests, sortie et dépendances. Sur GitHub : branche et PR revue ; Vercel preview après consentement.

Mesures : durée totale, conflits évités, nombre de corrections, coût estimé/reported, actions refusées, possibilité de reprise après interruption.

Critère de réussite de la démo : l'application stock démarre localement ; opérations CRUD + inventaire tests passent ; tous événements Nexus AI sont récupérables ; secrets non exposés ; le projet Git dispose d'un diff lisible, et la preview n'est produite que si l'architecture est compatible avec Vercel.

26. Risques, questions ouvertes et politique d'arbitrage

Risque/question	Décision par défaut / responsable
Nom Nexus AI déjà utilisé ou marque	nom provisoire ; vérification juridique avant communication publique
Runtime sandbox Windows/macOS	spike par OS, ne pas livrer autonomie sans tests ; responsable sécurité
Hermes CLI format/options exactes	connecteur en beta jusqu'à tests contre version réelle ; responsable intégration
Voice hors ligne multi-OS et taille binaire	comparaison moteurs avant choix ; responsable voice
Quotas abonnements ChatGPT/Claude	aucun accès API présumé ; vérifier conditions/documentation ; produit
Modèles/modes concurrent IA évoluent	manifest capabilities/version + tests contractuels
Clause open source et contributions	Apache-2.0 facilite services commerciaux mais autorise forks commerciaux ; examiner CLA/licence marque avant contributions externes
Extension Vercel pour backend non compatible	refuser déploiement auto, expliquer alternatives
Coût et latence de 5 agents	plafond par mission, mode 2 agents recommandé par défaut
Conflit entre exigences vitesse/sécurité	sécurité et protection secrets priment sur automatisation

27. Sources, documentation et révisions

Références consultées le 10 octobre 2026 :

- Electron Forge, template officiel Vite TypeScript : https://github.com/electron/forge/blob/main/docs/templates/vite-%2B-typescript.md
- Boilerplate alternatif Electron Vite React : https://github.com/electron-vite/electron-vite-react
- Hydra : https://github.com/jpdlr/hydra
- Runner : https://github.com/yicheng47/runner

- Codey : https://github.com/its-ahoh/codey
- Emdash : https://github.com/generalaction/emdash
- Pivio : https://pivio.app/
- Claude CLI : https://docs.anthropic.com/en/docs/claude-code/cli-usage
- Codex CLI : https://developers.openai.com/codex/cli/reference
- GitHub auth : https://cli.github.com/manual/gh_auth_login
- Vercel CLI : https://vercel.com/docs/projects/deploy-from-cli
- Apache-2.0 : https://www.apache.org/licenses/LICENSE-2.0.html

Gestion des versions : v0.1 vision ; v0.2 voix/GitHub/Vercel/sandbox/persistance ; v1.0 présent dossier de développement détaillé. Toute évolution requiert ticket, modification section, historique de version et, pour décision structurante, ADR. Ce document est une spécification de réalisation ; aucune fonctionnalité n'est déclarée déjà construite ou testée.